



Manual paso a paso

Autenticación federada, almacenamiento de tokens y CRUD real en la nube
Caso 2 — CampusLab

Asignatura: DSY1107 — Desarrollo Cloud Native I

Equipo: Katherine Ramírez · Marcelo Prado · Williams Rivas

Fecha: 13 de septiembre de 2026

Entorno demostrado: Frontend + Backend desplegados en AWS EC2 (HTTPS), Cognito federado con Microsoft Entra ID

1. Objetivo y alcance

Este manual documenta, con capturas reales del navegador y comandos de consola reales (no simulados), el funcionamiento completo del sistema CampusLab desplegado **en la nube** (no en localhost): el flujo de autenticación federada (Microsoft Entra ID → Amazon Cognito), el lugar exacto donde el navegador guarda el token de sesión sin cifrar, la evidencia de las llamadas reales a la API, y la verificación completa de las operaciones CRUD con control de roles.

Importante: todas las capturas de este manual fueron tomadas en vivo, en el mismo momento en que se ejecutaron las acciones descritas, contra la aplicación real desplegada en `https://3.235.133.3` (instancia EC2 `i-0448753bab6e8173b`), no contra un entorno de desarrollo local.

2. Preparación: publicar el frontend en la nube con HTTPS

Amazon Cognito exige que el *redirect URI* del flujo OAuth2/OIDC sea HTTPS o `localhost`; no acepta una IP pública en HTTP plano. Como la EC2 no tiene un dominio propio, se generó un certificado autofirmado y se publicó el build de producción del frontend con **nginx** directamente en la instancia.

2.1 Verificar el estado real de la instancia y el backend

```
$ aws ec2 describe-instances --instance-ids i-0448753bab6e8173b \
  --query 'Reservations[].Instances[].{State:State.Name,IP:PublicIpAddress}'
-----
|      IP      |  State  |
+-----+-----+
| 3.235.133.3 | running |
+-----+-----+

$ curl -s -m 5 http://3.235.133.3:8080/api/catalog
{"status":401,"error":"Unauthorized","message":"Autenticacion rechazada",
"reason":"Full authentication is required to access this resource"}
```

El backend (Spring Boot + Docker Compose) seguía activo tras varios días; solo cambió la IP pública de la EC2 (no tiene Elastic IP asociada), por lo que se actualizó la integración del API Gateway:

```
$ aws apigatewayv2 update-integration --api-id 3wwncomjkh \
  --integration-id f9e4e10 --integration-uri 'http://3.235.133.3:8080/{proxy}'
```

2.2 Certificado autofirmado y nginx en la EC2

```
$ sudo dnf install -y nginx
$ sudo openssl req -x509 -nodes -days 825 -newkey rsa:2048 \
  -keyout /etc/nginx/campuslab-certs/selfsigned.key \
  -out /etc/nginx/campuslab-certs/selfsigned.crt \
  -subj "/C=CL/ST=Metropolitana/L=Santiago/O=CampusLab/CN=3.235.133.3"

# server { listen 443 ssl; root /usr/share/nginx/campuslab-frontend; ... }
$ sudo nginx -t && sudo systemctl enable --now nginx
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

2.3 Registrar la URL HTTPS en Cognito y en Entra ID

Se agregó `https://3.235.133.3` como *callback/logout URI* válido en el App Client de Cognito, en el Security Group (puerto 443) y, crucialmente, en el registro de la aplicación en Microsoft Entra ID (sin este paso Microsoft rechaza el login con `AADSTS50011`):

```
$ az rest --method PATCH --uri "https://graph.microsoft.com/v1.0/applications/<objectId>" \
  --body '{"spa":{"redirectUris":["http://localhost:5173","https://3.235.133.3"]}}'
```

3. Autenticación federada — paso a paso con capturas reales

La aplicación usa un flujo de **dobles federación**: el usuario inicia sesión con su cuenta institucional de Microsoft (Entra ID) y, una vez autenticado, la aplicación intercambia esa identidad por un token de **Amazon Cognito** (Cognito federado con Entra ID como Identity Provider OIDC), que es el token real usado para llamar a la API.

3.1 Pantalla inicial — app real en la nube, HTTPS

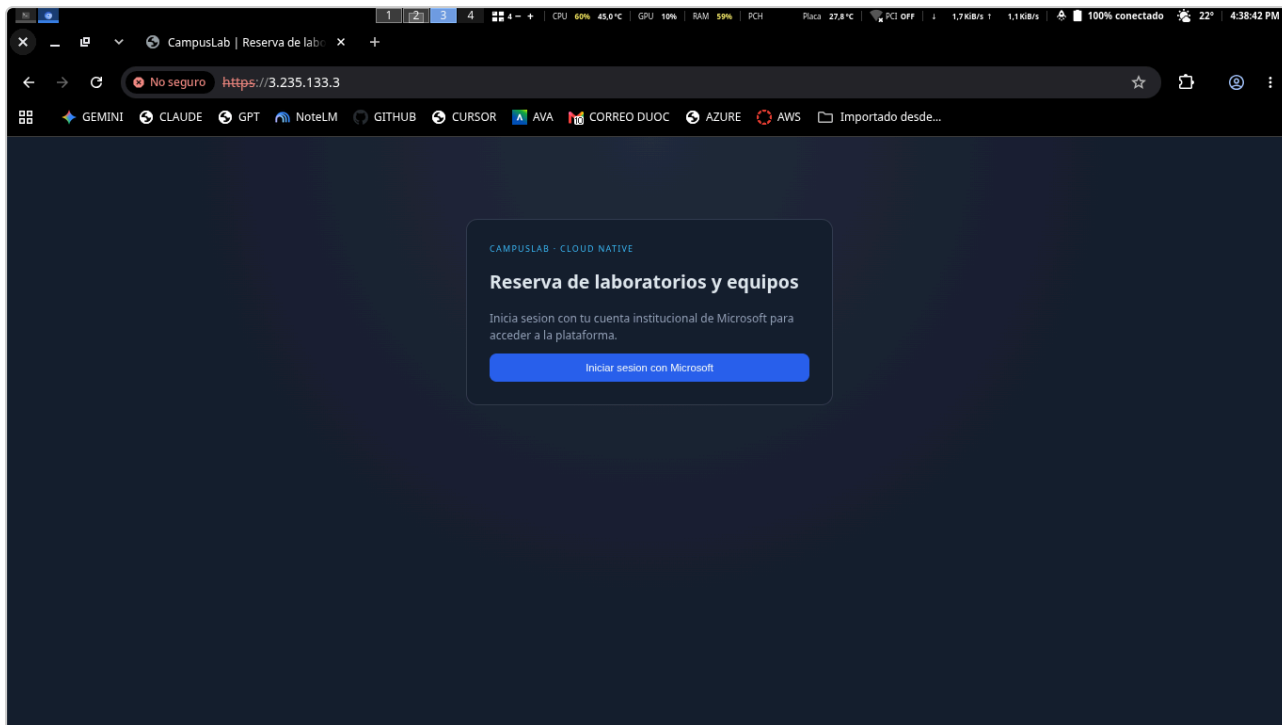


Figura 1. App CampusLab servida por nginx en la EC2 (<https://3.235.133.3>). El indicador "No seguro" es esperado: es un certificado autofirmado propio, no de una CA pública.

3.2 Paso 1 — Autenticación con Microsoft (Entra ID)

Al pulsar "Iniciar sesión con Microsoft", la app redirige a `login.microsoftonline.com` con el `client_id` de la app registrada en Entra ID. Como ya existía una sesión activa de Microsoft en el navegador, el login fue silencioso (SSO) y volvió directo a la app.

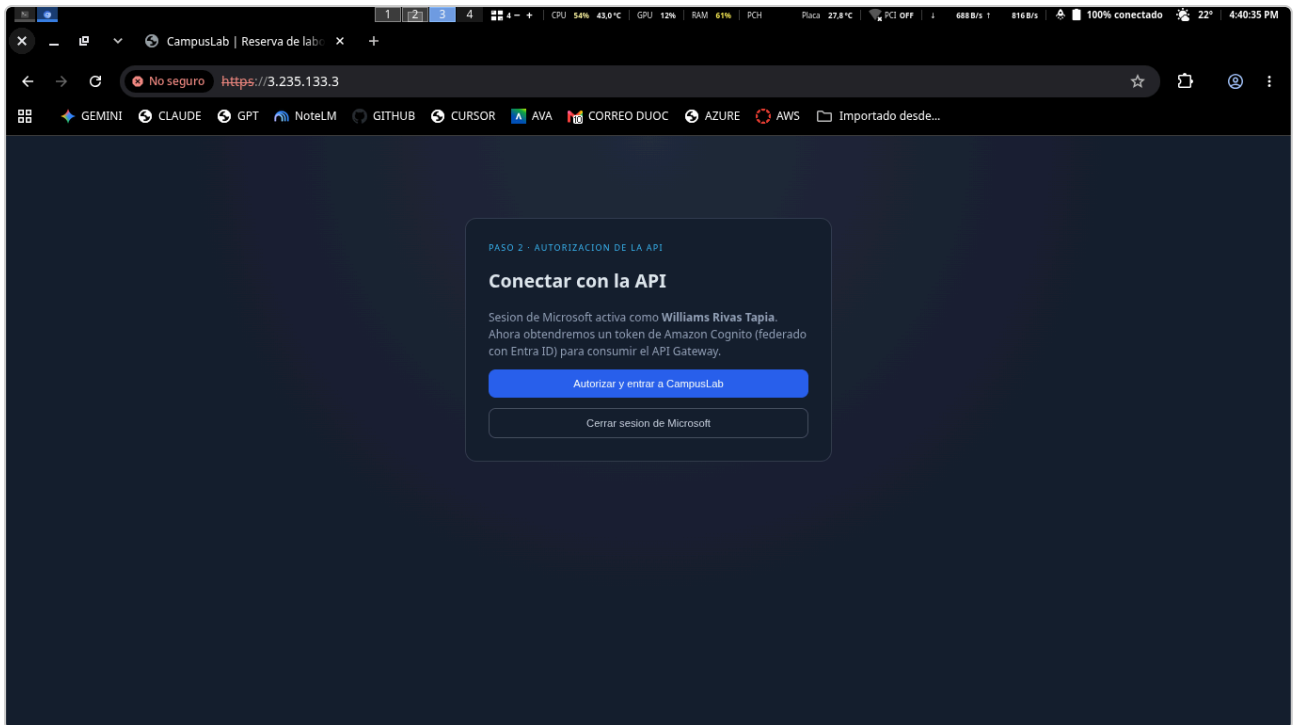


Figura 2. Paso 2 de la app: sesión de Microsoft activa confirmada ("Williams Rivas Tapia"), lista para pedir el token de Cognito.

3.3 Paso 2 — Autorización e intercambio por token de Cognito

Al pulsar "Autorizar y entrar a CampusLab", el navegador es redirigido al dominio de Cognito (`campuslab-wrivast.auth.us-east-1.amazoncognito.com`), que ya reconoce la identidad de Entra ID como Identity Provider federado y devuelve un token de acceso de Cognito real. La consola de operaciones carga el catálogo real desde la base de datos vía API Gateway → BFF → microservicio.

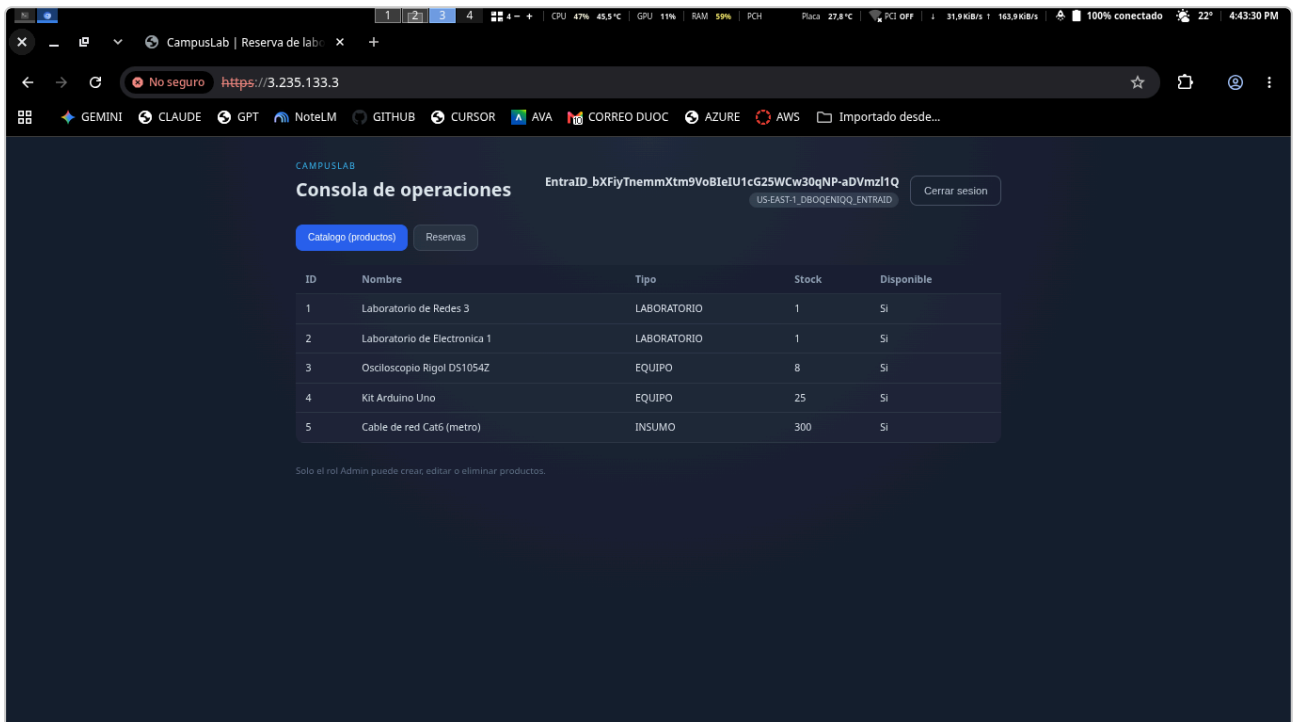


Figura 3. Sesión autenticada: usuario "EntraID_bxFiy..." con grupo "US-EAST-1_DBOQENIQQ_ENTRAID", catálogo real de 5 recursos cargado desde la base de datos.

Nota sobre el rol: el mensaje "Solo el rol Admin puede crear, editar o eliminar productos" es correcto y esperado: el usuario federado vía Entra ID cae en el grupo por defecto (sin privilegios de Admin). El rol Admin se demuestra en la sección 6 con la cuenta nativa de Cognito `demo.admin`.

El token de acceso de Cognito se guarda en **sessionStorage** del navegador, en texto plano (sin ningún cifrado adicional de la aplicación). Cualquier script con acceso al DOM de esa pestaña, o cualquier persona con acceso a las DevTools, puede leerlo directamente.

CampusLab | Reserva de laboratorio

No segurohttps://3.235.133.3

CAMPUSLAB

Consola de operaciones

EntralD_bXFiYtNmnmXtm9VoBieIU1cG25WCw30qNP-aDVmzl1Q

Cerrar sesión

Catálogo (productos)Reservas

ID	Nombre	Tipo	Stock	Disponible
1	Laboratorio de Redes 3	LABORATORIO	1	SI
2	Laboratorio de Electronica 1	LABORATORIO	1	SI
3	Osciloscopio Rigol DS1054Z	EQUIPO	8	SI
4	Kit Arduino Uno	EQUIPO	25	SI
5	Cable de red Cat6 (metro)	INSUMO	300	SI

Solo el rol Admin puede crear, editar o eliminar productos.

DevTools is now available in Spanish

Don't show againAlways match Chrome's languageSwitch DevTools to Spanish

Application

ManifestService workersStorageWebMCPNEW

Storage

Local storageSession storagehttps://3.235.133.3Extension storageIndexedDBCookiesPrivate state tokensInterest groupsShared storageCache storageStorage buckets

Background services

Back/forward cacheBackground fetchBackground syncBounce tracking mitigationsNotificationsPayment handlerPeriodic background syncSpeculative loadsPush messagingReporting APIDevice bound sessions

Frames

top

Filterhttps://3.235.133.3

Key	Value
cognito_access_token	eyJraWQ/Oj1WcG9QSEVnmMDZxZjR3p...

No value selected
Select a value to preview

4.2 Contenido completo del token (extraído en vivo desde el navegador)

eyJraWQiOiJlWG9QSEVmM0ZkXzRjd3pRcmllRHZRvzhqenArZjdwYTNO9mOUF4N2xzPSIsImFsZyI6IlJTMjU2In0.eyJzdWIiOiI0NDE4YjRmOC1mMDAxLTcwYzMtMTRKZS0wMTQxNjUyZjA3YTkiLCJjb2duaXRvOmduyb3VwcyciOiEwWyJ1cy1lYXN0LTFfZEJPuWVuaVFxX0VudHJsSUQiXSwiaXNzIjoiaHR0cHM6Ly9jb2duaXRvLWlkC51cy1lYXN0LEUeYWlhbm9uYXdzLmNvbS91cy1lYXN0LTFfZEJPuWVuaVFxIiwidmVyc2lvbiI6MiwiY2xpZW50X2lkIjoiiNnIzYXYy4dWNjOTZva2hhMHVqcGg5CHJJNGEilClJvcmlnaW5fanRlIjoyYTM1OTdhMtZmZkZS00ZDYSLTlhNTUtYTnjMGYN1jhmMAzIiwidG9rZXZ5fdXNlIjoieWYwbnJXXNziIiwic2NvcGUioiJvcGVuaWQgcHJvZmZlsSQBlbWFpbCIscmF1dGhfZGltdSI6MTc4ODQ1NywiZXhwIjoxeDZlc2MyMDUzLClpYXQioJe30dkzMjgONTcsImpoasiSi6Ijk2OWRmMjZjLWlxYTYtNGY5Ni05MjA4LU1TZtc3Njg4ZWJjOICisInVzZXJuYWI1IjoierWS0cmFRJRf9iWEZpeVRuZW1tWHRTOVZvQkl1SVUxY0cyNVdDdzMmcU5QLWFEVm16bDFRIIn0.ur2e9RVi1VZOVKj9oY9NP-U7DuHu0K1o3mwFzmB3ISyBSdj3CWb86n1Nu027Yos9sx3huAPgeJm9weOR3p20WvtF1aAbi-vF4AQPT7miwPCAzw_g4fhGat5Dzitxk_7NQCTETmv77BnxJQPtmuDkgUasYPakD1Y2iopZo6iROvnUGikrkR_00UHstR_w_glrFI0Zw-nDK9DVD3g7MmB2SyCRqsLqKrR7sqccaAOw7tHSztidu08elsmaosEUJYobmwA8ql6Tt72xs7uL7avJP8ERtZFhjDHIP0yaT-shzD29SKWA3yzwdzunU3YZrz3nSGOX7vIve4FmgU_OvDl-ssw

Un JWT tiene 3 partes separadas por puntos (header.payload.signature), codificadas en Base64 — **no encriptadas**, solo codificadas. Decodificando header y payload:

Header	Payload (claims)
<pre>{ "kid": "uXoPHEf3Fq+2kwzQrinDvQW8jzp +f7pa3N70f9Ax71s=", "alg": "RS256" }</pre>	<pre>{ "sub": "4418b4f8-f001-70c3-14de-0141652f07a9", "cognito:groups": ["us-east-1_dB0QeniQq_EntraID"], "iss": "https://cognito-idp.us-east-1.amazonaws.com/us-east-1_dB0QeniQq", "client_id": "6r3af8ucc96okla0ujph9pru4a", "token_use": "access", "scope": "openid profile email", "exp": 1789332057, "iat": 1789328457, "username": "EntraID_bXFiyTnemmxTm9VoBIeIU1cG25WCw30qNP-aDVmz11Q" }</pre>

Implicación de seguridad: como el token vive sin cifrar en sessionStorage, queda expuesto a ataques XSS (un script inyectado podría robarlo) — es el trade-off típico de un SPA sin backend-for-session; por eso el BFF valida `token_use` y `client_id` en cada request, y el token tiene vida corta (`exp - iat = 3600s` , visible en el payload).

5. Llamadas reales a la API

Toda petición del frontend a los datos pasa por **API Gateway** (`https://3wwncomjkh.execute-api.us-east-1.amazonaws.com`), que valida el JWT contra el *issuer* de Cognito antes de reenviar al backend (BFF) en la EC2. Prueba con el token real recién extraído del navegador:

```
$ curl -i https://3wwncomjkh.execute-api.us-east-1.amazonaws.com/api/catalog/resources \
-H "Authorization: Bearer eyJraWQiOiJlWG9Q...(token real de la sesión del navegador)"

HTTP/2 200
access-control-allow-origin: https://3.235.133.3
content-type: application/json

[{"id":1,"nombre":"Laboratorio de Redes 3","tipo":"LABORATORIO","stock":1,"disponible":true}},
...]
```

5.1 Bug real encontrado y corregido durante esta sesión: CORS preflight rechazado

Al probar el fetch real desde el navegador (no solo por curl), la consola mostró un error de CORS. Diagnóstico: la ruta única `ANY /{proxy+}` del API Gateway interceptaba también las peticiones `OPTIONS` (preflight) y las pasaba por el autorizador JWT, que las rechazaba con 401 porque el navegador nunca envía el token en el preflight:

```
$ curl -i -X OPTIONS .../api/catalog/resources -H "Origin: https://3.235.133.3" ...
HTTP/2 401 # <- ANTES del fix
{"message":"Unauthorized"}

# Fix: reemplazar la ruta "ANY" por rutas explícitas SIN incluir OPTIONS,
# dejando que el CORS automático de API Gateway responda el preflight:
$ aws apigatewayv2 create-route --api-id 3wwncomjkh --route-key "GET /{proxy+}" ...
$ aws apigatewayv2 create-route --api-id 3wwncomjkh --route-key "POST /{proxy+}" ...
$ aws apigatewayv2 delete-route --api-id 3wwncomjkh --route-id vs1yq1c # elimina "ANY"

$ curl -i -X OPTIONS .../api/catalog/resources -H "Origin: https://3.235.133.3" ...
HTTP/2 204 # <- DESPUES del fix, correcto
```

6. CRUD completo verificado (consola, con control de roles)

Secuencia completa ejecutada contra la API real, autenticando con las cuentas nativas de Cognito `demo.admin` y `demo.user` (`USER_PASSWORD_AUTH`), para demostrar además el control de autorización por rol (`cognito:groups`):

```

=====
0) Identidad AWS CLI
=====
{
  "UserId": "AROAUHAJTCMSNA2SSQT40:user3284908=w.rivas",
  "Account": "289930416932",
  "Arn": "arn:aws:sts::289930416932:assumed-role/voclabs/user3284908=w.rivas"
}

=====
1) GET catalog SIN token -> se espera 401 (API Gateway)
=====
HTTP 401
  body: {"message":"Unauthorized"}

=====
2) Obtener tokens JWT reales de Cognito (demo.admin / demo.user)
=====
admin token len: 1069   user token len: 1031
claims admin:
  "cognito:groups": [
  "client_id": "6r3af8ucc96okla0ujph9pru4a",
  "token_use": "access",
  "username": "demo.admin"

=====
3) GET catalog CON token admin -> 200
=====
HTTP 200
  body: [{"id":1,"nombre":"Laboratorio de Redes 3","tipo":"LABORATORIO","stock":1,"disponible":true}, {"id":2,"nombre":"Laboratorio de Electronica 1","tipo":"LABORATORIO","stock":1,"disponible":true}, {"id":3,"nombre":"Osciloscopio Rigol DS1054Z","tipo":"EQUIPO","stock":8,"disponible":true}, {"id":4,"nombre":"Kit Arduino Uno","tipo":"EQUIPO","stock":25,"disponible":true}, {"id":5,"nombre":"Cable de red Cat6 (

=====
4) POST catalog como ADMIN -> 201 Created
=====
HTTP 201
  body: {"id":6,"nombre":"Laboratorio de IoT","tipo":"LABORATORIO","stock":1,"disponible":true}

=====
5) POST catalog como USUARIO (sin rol Admin) -> 403 Forbidden (BFF)
=====
HTTP 403
  body:

=====
6) POST bookings con RUT invalido y sin nombre -> 400 (validacion de esquema)
=====
HTTP 400
  body: {"error":"Bad Request","message":"La validacion de esquema fallo","errores":{"rut":"El rut debe tener formato 12345678-9 (sin puntos)","nombre":"El nombre es obligatorio"},"status":400}

=====
7) POST bookings valido como ADMIN -> 201 (SOLICITADA)
=====
HTTP 201
  body: {"id":3,"nombre":"Reserva demo","rut":"12345678-9","fecha":"2026-09-20T10:30:00","productId":1,"estado":"SOLICITADA"}

=====

```

FIN – resumen esperado: 401 . 200 . 201 . 403 . 400 . 201
=====

Paso	Código HTTP	Qué demuestra
Sin token	401	API Gateway exige autenticación
GET catálogo (admin)	200	Lectura autorizada
POST catálogo (admin)	201	Creación autorizada por rol Admin
POST catálogo (usuario)	403	Autorización por rol rechaza a no-Admin
POST reserva inválida	400	Validación de esquema (RUT/nombre)
POST reserva válida (admin)	201	Creación exitosa, estado SOLICITADA

7. Federación Cognito ↔ Entra ID (dónde está configurada)

La federación se configura en dos puntos que deben coincidir: el **Identity Provider OIDC** dentro del User Pool de Cognito, y el **registro de aplicación** en Microsoft Entra ID. Evidencia real de ambos lados:

```
=== Cognito: proveedor de identidad federado (Entra ID) ===
{
  "ProviderName": "EntraID",
  "ProviderType": "OIDC",
  "Issuer": "https://login.microsoftonline.com/d84b2661-a954-4b42-a9af-6b9c4990fb96/v2.0",
  "ClientId": "7c3d6c13-d08e-4a6f-a3dd-59ae9a171ac8",
  "AttributeMapping": {
    "email": "email",
    "username": "sub"
  }
}

=== Cognito: proveedores soportados en el App Client ===
[
  "COGNITO",
  "EntraID"
]

=== Entra ID: registro de la app federada (Azure AD / Microsoft Graph) ===
{
  "appId": "7c3d6c13-d08e-4a6f-a3dd-59ae9a171ac8",
  "displayName": "CampusLab-Cognito",
  "spaRedirectUris": [
    "https://3.235.133.3/",
    "https://3.235.133.3",
    "http://localhost:5173"
  ],
  "webRedirectUris": [
    "https://campuslab-wrivast.auth.us-east-1.amazoncognito.com/oauth2/idpresponse"
  ]
}

=== Entra ID: tenant y usuario autenticado ===
{
  "tenant": "d84b2661-a954-4b42-a9af-6b9c4990fb96",
  "usuario": "w.rivas@duocuc.cl"
}
```

Cómo funciona: Cognito conoce el *issuer* OIDC de Entra ID (`login.microsoftonline.com/<tenant>/v2.0`) y el `client_id` de la app registrada; Entra ID, a su vez, tiene registrada la URL de callback de Cognito (`.../oauth2/idpresponse`) como *redirect URI* válido. Cuando el usuario se autentica en Microsoft, Entra ID redirige a Cognito con un código de autorización; Cognito lo cambia por la identidad del usuario y emite su propio JWT, mapeando el usuario federado al grupo `us-east-1_dB0QeniQq_EntraID` (visible en el claim `cognito:groups` del token, sección 4).

8. Resumen

Aspecto	Evidencia
App en la nube (no localhost)	Figura 1, HTTPS en EC2 3.235.133.3
Autenticación federada	Figuras 2 y 3, login Microsoft → token Cognito
Token sin cifrar	Figura 4 + valor completo decodificado (sección 4)
Llamados reales a la API	curl con token real, sección 5
CRUD completo + roles	Secuencia 401·200·201·403·400·201, sección 6
Federación Cognito ↔ Entra	Comandos CLI de ambos lados, sección 7

Manual generado el 13 de septiembre de 2026 — Caso 2 CampusLab, DSY1107 Desarrollo Cloud Native I, DuocUC. Todas las capturas y salidas de consola corresponden a acciones ejecutadas en vivo contra el entorno real en AWS.